

# Lecture 7: Non-parametric regression, additive models

Max G'Sell  
Machine Learning I: 46-926

November 18, 2020

# Announcements

- ▶ Homework 3 is due today.
- ▶ No homework due Thanksgiving week, no quiz during Thanksgiving break. Instead, there will be a two-week homework. The first part will come out this week (~Thursday), the second part will come out next week.
- ▶ Some comments and adjustments to quiz format.

# Today: Cross-validation, starting nonparametrics

- ▶ A few comments on quizzes in general
- ▶ A discussion of the homework
- ▶ Less biased than linear regression: non-parametrics
  - ▶ Kernel regression, quickly
  - ▶ Splines
  - ▶ Additive models

## Review: What next?

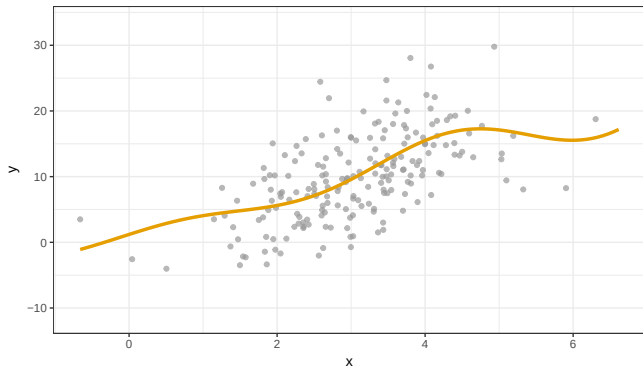
We started out by noting that we want to approximate  $\mathbb{E}(Y|X = x)$ .

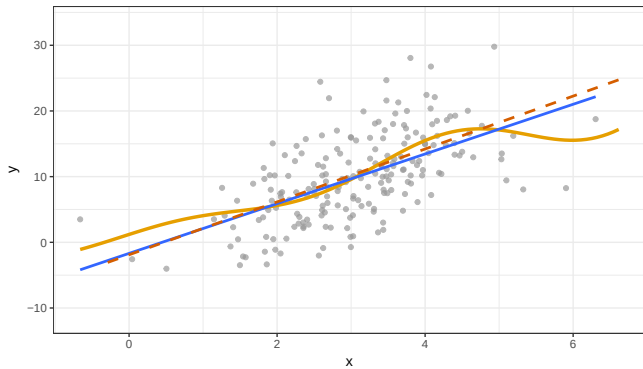
We thought about linear approximations,  $\mathbb{E}(Y|X = x) = x^T \beta$ , and showed that we can beat the least squares estimate with regularization to

- ▶ Reduce the variance (at the cost of bias).
- ▶ Improve interpretability by adding sparsity

However, at the end of the day, we're still using something linear!

How do we move to something with lower bias instead of lower variance?





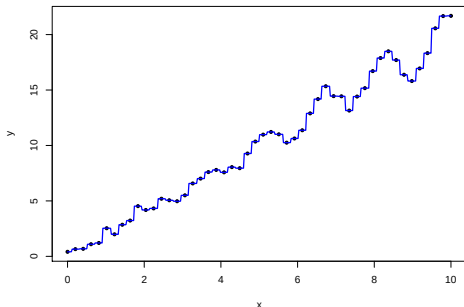
# Nonparametric regression

We will talk about two overall approaches to nonlinear regression. We begin with  $X \in \mathbb{R}$ , the one dimensional problem shown in the pictures above.

**Kernel-based methods:** We use a version of local averaging to approximate the function. Includes kernel smoothing, local linear regression. Closely related to  $k$ -nearest neighbors.

**Spline-based methods.** We fit a flexible piecewise polynomial to the data in a way that compromises between goodness-of-fit and smoothness.

## $k$ -nearest neighbors



Recall that we want  $\hat{\mu}(x)$  to be a good approximation of  $\mathbb{E}(Y|X = x)$ . What if we don't want to assume a strong form for the  $\mathbb{E}(Y|X = x)$  function?

We might still be willing to assume that points near  $x$  are similar to  $x$ . We can use this to estimate the conditional mean!



## Kernel regression

$k$ -nearest neighbors can perform very well (in low dimensions or with nice distances). However, we might be bothered by how jagged the fit is!

This happens because being one of the  $k$ -nearest points is a jumpy idea. We can switch to a smoother notion of a neighborhood to get a smoother estimate.

## Kernel regression

$$\hat{\mu}(x) = \sum_{i=1}^n y_i \frac{K(x_i, x)}{\sum_{j=1}^n K(x_j, x)}$$

$$K(x, x_j) = \frac{1}{\sqrt{2\pi h}} e^{-(x-x_j)^2/2h} \quad (\text{Gaussian, example})$$

Kernel regression gives a very simple way to get non-linear estimates of a function.

As we tune  $h$ , we determine how locally the fit looks at the training points, trading off bias and variance!

The method also has the advantage of being:

- ▶ Easy to prove things about
- ▶ Easy to generalize to other dimensions
- ▶ Easy to generalize to other methods

## Kernel regression, example result

$$\begin{aligned}MSE(x) = & \sigma^2(x) + h^4 \left( \frac{1}{2} \mu''(x) + \frac{\mu'(x) f'(x)}{f(x)} \right)^2 (\sigma_K^2)^2 \\ & + \frac{\sigma^2(x) R(K)}{nh f(x)} + o(h^4) + o((nh)^{-1})\end{aligned}$$

Some simplification (and hiding):

$$MSE(x) = \sigma^2(x) + O(h^4) + O((nh)^{-1})$$

## Kernel regression, higher dimensions

Suppose that we now have more than one dimension of  $X$ . How do we do nonparametric regression?

Just define a kernel to measure higher dimensional similarity!

$$K(x - x_j) = K_1(x^1 - x_j^1)K_2(x^2 - x_j^2) \cdots K_d(x^d - x_j^d)$$

Now everything works as before! Note that each kernel piece can have its own bandwidth. . .

However, we adapt terribly as dimension increases!

$$MSE - \sigma^2 =$$

## Kernel regression: local linear regression

We can extend the idea of kernel regression, which just looks at local averages, to other local functions.

A classic example is local linear regression, which you have seen.

$$\min_{\beta_0, \beta} \frac{1}{n} \sum_{i=1}^n w_i(x) (y_i - \beta_0 - (x_i - x)\beta)^2$$

This is just weighted linear regression, so we already know the answer.

This leads to a smoother fit, can extrapolate better, and corrects a small term in the bias calculation.

## An alternative approach: smoothing splines

Instead of kernel smoothing, suppose we started by trying to build a flexible function that:

- ▶ Approximates our data well.
- ▶ Is smooth.

We could try to encode this as an optimization problem:

$$\hat{f} = \operatorname{argmin}_f$$

over twice-differentiable  $f$ .

# Smoothing splines

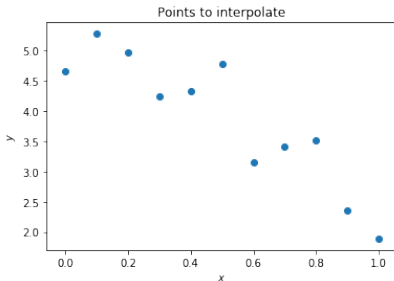
This is just another penalized regression problem!

$$\hat{f} = \operatorname{argmin}_f \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int f''(x)^2 dx$$

- ▶ As  $\lambda$  gets larger:
- ▶ If  $\lambda \rightarrow \infty$ :
- ▶ As  $\lambda$  gets smaller:
- ▶ As  $\lambda \rightarrow 0$ :

## Digression: splines

Splines come out of function approximation and interpolation. Suppose that I wanted to represent a wiggly function, or just interpolate some points.



Kernel regression is very expensive to evaluate. A linear model isn't going to do a great job.

I could add some wiggle with a higher order polynomial. . .

**[Switch to python interpolation notebook]**



# Splines

Cubic splines give a very nice solution to the interpolation problem.

The function between points is modeled as piecewise cubic. This gives well-behaved wiggle.

The function is constrained to have continuous  $f(x)$ ,  $f'(x)$ ,  $f''(x)$  at the points where cubics change.

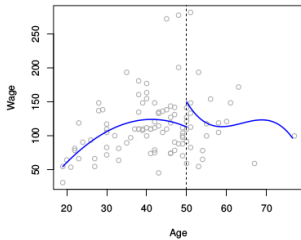
Despite the apparent complexity, we can write a  $K$ -knot spline as

$$f(x) = \beta_0 + \beta_1 b_1(x) + \beta_2 b_2(x) + \cdots + \beta_{K+3} b_{K+3}(x)$$

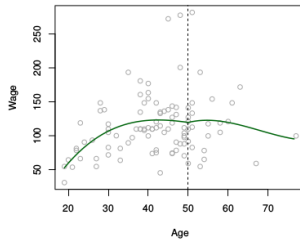
for very well-behaved bases  $b_1, \dots, b_{K+3}$ , making computation fast and efficient!

# Splines

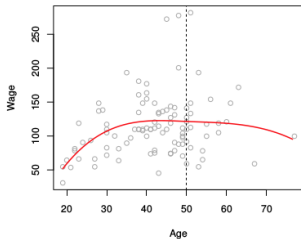
**Piecewise Cubic**



**Continuous Piecewise Cubic**



**Cubic Spline**



## Smoothing splines

Back to our problem:

$$\hat{f} = \operatorname{argmin}_f \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int f''(x)^2 dx$$

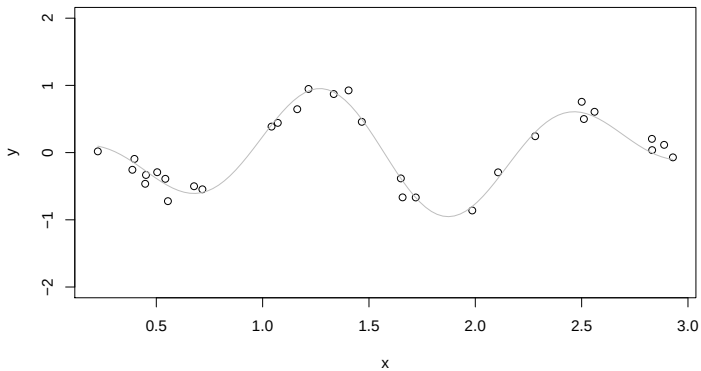
This looks like a pretty ugly optimization problem. However, the solution turns out to be very nice!

The minimizer is always a **cubic spline**!

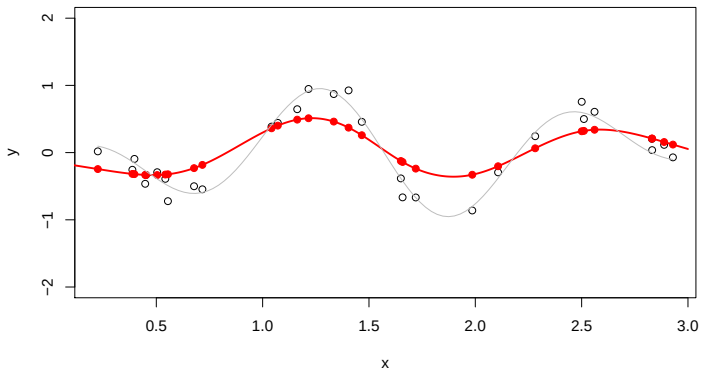
Furthermore, fitting the cubic spline is a linear operator on  $Y$ !

We can steal all the work done over the years in applied mathematics, making computation convenient and efficient.

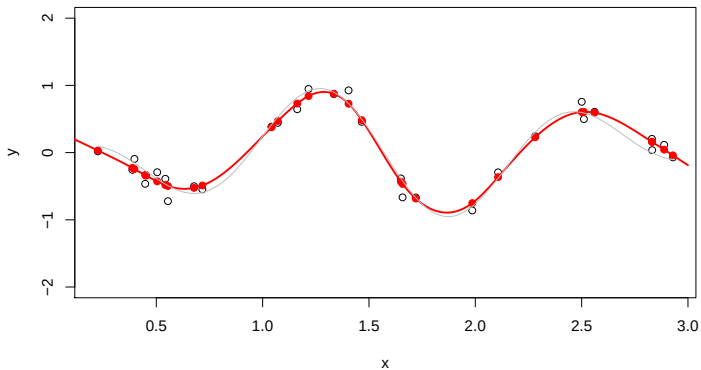
# Smoothing splines



# Smoothing splines



# Smoothing splines



## Smoothing splines

$$\hat{f} = \operatorname{argmin}_f \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int f''(x)^2 dx$$

This trick gives a computationally efficient way to swap smooth, flexible functions in for linear functions throughout our methods.

We'll see several applications of this in the course.

## Now what?

So far, we've seen that:

- ▶ We can build nonlinear function estimates that converge to the true regression function with enough data (and smoothness).
- ▶ We can make them efficient in one dimension, using splines.
- ▶ Our kernel methods break down in higher dimensions.

We don't usually just care about one-dimensional regression! What can we do?



## More than one dimension

Suppose that we want to estimate  $Y$  from  $X \in \mathbb{R}^p$ , with  $p > 1$ .

For quite small  $p$ , we could still use kernel regression. There are also two generalizations of splines to low dimensions, but they suffer from the same curse.

We look for a compromise: more flexibility than linear regression, but less than general  $d$ -dimensional function estimation.

## Additive models

Think about our linear models:

$$\hat{f}(x) = \beta_0 + \sum_{j=1}^p \hat{\beta}_j X_j$$

There are really two modeling choices going into this:

- ▶ Additivity: The each feature contributes additively to the prediction.
- ▶ Linearity: The values of each feature enter linearly.

It turns out that linearity can be relaxed without sacrificing much efficiency!

## Additive models

Additive models:

$$\hat{f}(x) = \beta_0 + \sum_{j=1}^p \hat{f}_j(X_j)$$

We can choose the  $\hat{f}_j$  in many different ways. We can even vary the choice for each feature!

- ▶ Linear function
- ▶ Polynomial function
- ▶ Smoothing spline
- ▶ Kernel regression
- ▶ Local linear fit

In practice, fitting smoothing splines for each  $\hat{f}_j$  is most common. This is what the `LinearGAM` function in python does (package `pygam`, and what `gam`, `mgcv` packages do in R. (*Note: mgcv is the easiest.*)

## Additive models

To estimate a function linearly, we expect

$$MSE =$$

To estimate a 1-dimensional function nonparametrically, we expect

$$MSE =$$

To estimate a  $d$ -dimensional function nonparametrically, we expect

$$MSE =$$

## Additive models

To estimate a  $d$ -dimensional function nonparametrically, we expect

$$MSE = O\left(\frac{1}{n^{4/(d+4)}}\right)$$

To estimate a  $d$ -dimensional *additive* model, we just estimate each piece separately, giving

$$MSE =$$

This means that, if we don't care about this minor drop in convergence rate, we can replace linear regression with non-linear functions of each variable!

No more guessing at variable transformations by hand!

## What's next

We've talked about several topics for prediction, starting with linear regression and moving in both lower variance and lower bias directions.

We're going to change gears and talk about classification, the other major chunk of supervised learning.

After we're comfortable with both of these areas, we're going to talk about more advanced modeling techniques that are useful for both prediction and classification.